

FIȘĂ DE LUCRU

Subprograme (funcții) în Python

Disciplina: Informatică	Clasa: a IX-a (mate-info)	Durata: ~50 min
--------------------------------	----------------------------------	------------------------

Nume și prenume elev: Data:

Ce vei învăța: ce este un subprogram și de ce este util; cum definești și cum apelezi un subprogram în Python (def, parametri, return); cum funcționează apelul; cum folosești subprograme predefinite (abs, round, sqrt, len, min, max, sum).

1. Ce este un subprogram?

Un **subprogram** (numit în Python **funcție**) este o secvență de instrucțiuni cu un nume, care rezolvă o sarcină bine definită și poate fi **apelată (folosită) ori de câte ori avem nevoie**, fără a rescrie codul.

Analogie: Un subprogram este ca o rețetă de bucătărie. O scrii o singură dată („Cum fac o omletă”) și apoi o poți folosi de câte ori vrei, doar spunând „fă o omletă”, fără să reexplici toți pașii de fiecare dată.

2. De ce folosim subprograme?

- Reutilizare: scrii codul o singură dată și îl folosești de mai multe ori.
- Cod mai clar și mai scurt: programul devine ușor de citit și de înțeles.
- Modularizare: împărțim o problemă mare în sarcini mici, ușor de rezolvat.
- Depanare mai ușoară: dacă apare o eroare, o căutăm într-un singur loc.
- Lucru în echipă: fiecare coleg poate scrie câte un subprogram.

3. Structura unui subprogram în Python

Definirea unui subprogram începe cu cuvântul-cheie **def**, urmat de nume, lista de parametri între paranteze și caracterul **:**. Corpul este **indentat** (deplasat spre dreapta).

```
def nume_subprogram(parametru1, parametru2):  
    # corpul subprogramului (indentat)  
    rezultat = parametru1 + parametru2  
    return rezultat          # returnează un rezultat (opțional)
```

Componente:

- Antet: linia `def nume(parametri):` — stabilește numele și datele de intrare.
- Parametri: datele de intrare primite de subprogram (pot lipsi).
- Corp: instrucțiunile indentate care realizează sarcina.
- return: trimite înapoi un rezultat la locul apelului (opțional).

4. Apelul unui subprogram. Cum funcționează?

Un subprogram **definit** nu se execută singur. El se execută doar când este **apelat**, scriindu-i numele urmat de paranteze cu *argumente*:

```
def suma(a, b):  
    return a + b
```

```
rezultat = suma(3, 5)    # apel: a primește 3, b primește 5
print(rezultat)          # afișează 8
```

Mecanismul de execuție: (1) la apel, argumentele 3 și 5 sunt transmise parametrilor a și b; (2) se execută corpul subprogramului; (3) instrucțiunea return trimite înapoi valoarea 8; (4) execuția continuă din locul apelului.

Atenție: parametrii sunt numele din definiție (a, b), iar argumentele sunt valorile efective de la apel (3, 5).

5. Variabile locale și globale

- O variabilă locală există doar în interiorul subprogramului în care a fost creată.
- O variabilă globală este definită în programul principal și poate fi citită din mai multe locuri.
- Recomandare: folosește parametri și return în loc de variabile globale — codul devine mai sigur și mai clar.

6. Subprograme predefinite (deja existente în Python)

Python pune la dispoziție multe subprograme gata făcute. Unele sunt direct disponibile, altele se află în biblioteci (de exemplu math).

Funcție	Ce face	Exemplu
<code>abs(x)</code>	valoarea absolută (modulul)	<code>abs(-7) → 7</code>
<code>round(x, n)</code>	rotunjire la n zecimale	<code>round(3.14159, 2) → 3.14</code>
<code>math.sqrt(x)</code>	radicalul (rădăcina pătrată)	<code>math.sqrt(25) → 5.0</code>
<code>len(c)</code>	numărul de elemente	<code>len([4, 8, 1]) → 3</code>
<code>min(c) / max(c)</code>	minimul / maximul	<code>max([4, 8, 1]) → 8</code>
<code>sum(c)</code>	suma elementelor	<code>sum([4, 8, 1]) → 13</code>

Pentru a folosi `math.sqrt` scriem la început: `import math`

7. Exemple rezolvate pas cu pas

Exemplul 1. Subprogram fără parametri

Cerință: Scrie un subprogram care afișează un mesaj de salut.

```
def saluta():
    print("Bună ziua! Bine ai venit la ora de informatică.")

saluta()          # apelul afișează mesajul
```

Explicație: Subprogramul `saluta()` nu primește date și nu returnează nimic; doar afișează un mesaj de fiecare dată când este apelat.

Exemplul 2. Subprogram cu parametri și return

Cerință: Scrie un subprogram care calculează aria unui dreptunghi cu laturile L și l.

```
def arie_dreptunghi(L, l):
    return L * l

a1 = arie_dreptunghi(5, 3)    # 15
a2 = arie_dreptunghi(10, 2)   # 20
print(a1, a2)                 # 15 20
```

Explicație: Definim formula o singură dată. La fiecare apel transmitem alte valori pentru L și l și obținem alt rezultat. Astfel evităm rescrierea formulei.

Exemplul 3. Subprogram care returnează True/False

Cerință: Scrie un subprogram care verifică dacă un număr este par.

```
def este_par(n):  
    return n % 2 == 0  
  
print(este_par(8))    # True  
print(este_par(5))    # False
```

Explicație: Operatorul % dă restul împărțirii. Dacă restul la împărțirea cu 2 este 0, numărul este par, deci subprogramul returnează True.

Exemplul 4. Folosirea subprogramelor predefinite

Cerință: Calculează media a trei note și afișează-o rotunjită la 2 zecimale, folosind funcții predefinite.

```
note = [9, 7, 10]  
media = sum(note) / len(note)  
print(round(media, 2))    # 8.67
```

Explicație: Folosim sum() pentru a aduna notele, len() pentru a afla câte sunt și round() pentru a rotunji rezultatul. Nu a trebuit să scriem noi aceste subprograme.

8. Aplicații propuse

Rezolvă pe calculator. Scrie codul și testează-l cu mai multe valori.

1. Scrie un subprogram suma(a, b) care returnează suma a două numere. Apelează-l pentru perechile (4, 9) și (12, 8).
2. Scrie un subprogram patrat(n) care returnează n^2 . Afișează pătratele numerelor de la 1 la 5 (folosind un apel într-o structură repetitivă).
3. Scrie un subprogram saluta_personalizat(ume) care afișează „Salut, <nume>!”. Apelează-l cu numele tău.
4. Scrie un subprogram maxim2(a, b) care returnează cel mai mare dintre două numere (fără a folosi max). Verifică-l cu min/max predefinite.
5. Scrie un subprogram perimetru_patrat(latura) și folosește-l pentru a calcula perimetrul a două pătrate diferite.
6. (Provocare) Scrie un subprogram este_prim(n) care returnează True dacă n este număr prim, altfel False.

Reține!

Definire: def nume(parametri): ... return rezultat • Apel: nume(argumente) • return trimite înapoi un rezultat; fără return, subprogramul doar execută instrucțiuni (de ex. afișează).